architectures, and a brief history of ARMv7 and the x86. We also review the controversial subjects of high-level-language computer architectures and reduced instruction set computer architectures. The history of programming languages includes Fortran, Lisp, Algol, C, Cobol, Pascal, Simula, Smalltalk, C++, and Java, and the history of compilers includes the key milestones and the pioneers who achieved them. The rest of **Section 2.22** is found online.

2.23 Exercises

(Readers may wish to download and install a free Community Edition of the ARM DS-5 professional software suite which contains an ARMv8-A (64-bit) architecture simulator before attempting these exercises. More details are available on the companion Web site for this book.)

The main difference between LEGv8 and ARMv8, is that LEGv8 is a subset of ARMv8. Thus, compilers may generate ARMv8 instructions that are not found in LEGv8. That is one reason we list the rest of the ARMv8 instructions in Sections 2.19, 3.8, and 5.14. If you were to look at the output of a compiler, the most obvious difference would be the likely use of scaled loads and stores (`LDR` and `STR`) instead of the unscaled versions that we feature in LEGv8 (`LDUR` and `STUR`).

We also made some changes to ARMv8 for pedagogic reasons in LEGv8, which we noted in the *Elaboration* sections in the text, that will not work with the ARMv8 assembler and simulators. Here are the four differences:

- Immediate versions are not separate assembly language instructions in ARMv8. While LEGv8 has `ADDI`, `SUBI`, `ADDIS`, `SUBIS`, `ANDI`, `ORRI`, and `EORI`, ARMv8 does not. You simply write the non-immediate version and use an immediate operand. For example, the ARMv8 assembler accepts

  ```
  ADD X0, X1, #4
  ```

  and produces the immediate version of the opcode for `ADD`, which LEGv8 associates with `ADDI`.

- Similarly, single- and double-precision floating-point instructions do not have separate assembly language instructions. Thus, ARMv8 does not have `FADDS`, `FADDD`, `FSUBS`, `FSUBD`, `FMULS`, `FMULD`, `FDIVS`, `FDIVD`, `FCMPS`, `FCMPD`, `LDURS`, `LDURD`, `STURS`, and `STURD`, as does LEGv8. Instead, you just write the floating-point operation, and the assembler produces the right

opcode depending on which register name you use: S for single and D for double. For example, the ARMv8 assembler accepts

```
FADD D0, D1, D2
```

and produces the double-precision version of the opcode for `FADD`, which LEGv8 associates with `FADDD`.

- In the full ARMv8 instruction set, register 31 is `XZR` in most instructions but the *stack pointer* (`SP`) in others. We think it is confusing, so register 31 is always `XZR` and `SP` is always register 28 in LEGv8. If you stick to using `XZR` and `SP` for register names when using the assembler and simulator, it should not be a problem, but this subtle distinction might show up in visualizations of the state of the processor, for example.

- The immediate fields for `ANDI`, `ORRI`, and `EORI` of the full ARMv8 instruction set are not simple 12-bit immediates that we assume in LEGv8. ARMv8 has an algorithm for encoding immediate values as repeating patterns. This means that some small constants (e.g., 1, 2, 3, 4, and 6) are valid, while others (e.g., 0, 5) are not. Thus, the assembler may insert more instructions than you might expect to create simple constants, or fewer if you pick a lucky large constant. The official definition is the bit pattern can be viewed "as a vector of identical elements of size e = 2, 4, 8, 16, 32, or 64 bits. Each element contains the same sub-pattern, that is a single run of 1 to (e − 1) nonzero bits from bit 0 followed by zero bits, then rotated by 0 to (e − 1) bits." Don't try to figure this out yourself; just leave it to the assembler!

2.1 [5] <§2.2> For the following C statement, write the corresponding LEGv8 assembly code. Assume that the C variables `f`, `g`, and `h`, have already been placed in registers `X0`, `X1`, and `X2` respectively. Use a minimal number of LEGv8 assembly instructions.

```
f = g + (h - 5);
```

2.2 [5] <§2.2> Write a single C statement that corresponds to the two LEGv8 assembly instructions below.

```
ADD f, g, h
ADD f, i, f
```

2.3 [5] <§§2.2, 2.3> For the following C statement, write the corresponding LEGv8 assembly code. Assume that the variables `f`, `g`, `h`, `i`, and `j` are assigned to registers `X0`, `X1`, `X2`, `X3`, and `X4`, respectively. Assume that the base address of the arrays `A` and `B` are in registers `X6` and `X7`, respectively.

```
B[8] = A[i-j];
```

2.4 [10] <§§2.2, 2.3> For the LEGv8 assembly instructions below, what is the corresponding C statement? Assume that the variables `f`, `g`, `h`, `i`, and `j` are assigned to registers `X0`,`X1`,`X2`,`X3`, and `X4`, respectively. Assume that the base address of the arrays A and B are in registers `X6` and `X7`, respectively.

```
LSL  X9, X0, #3     // X9 = f*8
ADD  X9, X6, X9     // X9 = &A[f]
LSL  X10, X1, #3    // X10 = g*8
ADD  X10, X7, X10   // X10 = &B[g]
LDUR X0, [X9, #0]   // f = A[f]

ADDI X11, X9, #8
LDUR X9, [X11, #0]
ADD  X9, X9, X0
STUR X9, [X10, #0]
```

2.5 [5] <§§2.2, 2.3, 2.6> For the LEGv8 assembly instructions in Exercise 2.4, rewrite the assembly code to minimize the number of LEGv8 instructions needed to carry out the same function.

2.6 [5] <§2.3> Show how the value `0xabcdef12` would be arranged in memory of a little-endian and a big-endian machine. Assume the data are stored starting at address 0 and that the word size is 4 bytes.

2.7 [5] <§2.4> Translate `0xabcdef12` into decimal.

2.8 [5] <§§2.2, 2.3> Translate the following C code to LEGv8. Assume that the variables `f`, `g`, `h`, `i`, and `j` are assigned to registers `X0`,`X1`,`X2`,`X3`, and `X4`, respectively. Assume that the base address of the arrays `A` and `B` are in registers `X6` and `X7`, respectively. Assume that the elements of the arrays `A` and `B` are 8-byte words:

```
B[8] = A[i] + A[j];
```

2.9 [10] <§§2.2, 2.3> Translate the following LEGv8 code to C. Assume that the variables `f`, `g`, `h`, `i`, and `j` are assigned to registers `X0`,`X1`,`X2`,`X3`, and `X4`, respectively. Assume that the base address of the arrays `A` and `B` are in registers `X6` and `X7`, respectively.

```
ADDI X9,  X6,  #8
ADD  X10, X6,  XZR
STUR X10, [X9, #0]
LDUR X9,  [X9, #0]
ADD  X0,  X9,  X10
```

2.10 [20] <§§2.2, 2.5> For each LEGv8 instruction in Exercise 2.9, show the value of the opcode (Op), source register (Rn), and target register (Rd or Rt) fields. For the I-type instructions, show the value of the immediate field, and for the R-type instructions, show the value of the second source register (Rm).

2.11 Assume that registers `X0` and `X1` hold the values `0×8000000000000000` and `0×D000000000000000`, respectively.

2.11.1 [5] <§2.4> What is the value of `X9` for the following assembly code?

```
ADD X9, X0, X1
```

2.11.2 [5] <§2.4> Is the result in `X9` the desired result, or has there been overflow?

2.11.3 [5] <§2.4> For the contents of registers `X0` and `X1` as specified above, what is the value of `X9` for the following assembly code?

```
SUB X9, X0, X1
```

2.11.4 [5] <§2.4> Is the result in `X9` the desired result, or has there been overflow?

2.11.5 [5] <§2.4> For the contents of registers `X0` and `X1` as specified above, what is the value of `X9` for the following assembly code?

```
ADD X9, X0, X1
ADD X9, X9, X0
```

2.11.6 [5] <§2.4> Is the result in `X9` the desired result, or has there been overflow?

2.12 Assume that `X0` holds the value 128_{ten}.

2.12.1 [5] <§2.4> For the instruction `ADD X9,X0,X1`, what is the range(s) of values for `X1` that would result in overflow?

2.12.2 [5] <§2.4> For the instruction `SUB X9,X0,X1`, what is the range(s) of values for `X1` that would result in overflow?

2.12.3 [5] <§2.4> For the instruction `SUB X9,X1,X0`, what is the range(s) of values for `X1` that would result in overflow?

2.13 [5] <§§2.2, 2.5> Provide the instruction type and assembly language instruction for the following binary value:

$$1000\ 1011\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000_{two}$$

Hint: Figure 2.20 may be helpful.

2.14 [5] <§§2.2, 2.5> Provide the instruction type and hexadecimal representation of the following instruction:

```
STUR X9, [X10,#32]
```

2.15 [5] <§2.5> Provide the instruction type, assembly language instruction, and binary representation of instruction described by the following LEGv8 fields:

```
op=0x658, Rm=13, Rn=15, Rd=17, shamt=0
```

2.16 [5] <§2.5> Provide the instruction type, assembly language instruction, and binary representation of instruction described by the following LEGv8 fields:

```
op=0×7c2, Rn=12, Rt=3, const=0×4
```

2.17 Assume that we would like to expand the LEGv8 register file to 128 registers and expand the instruction set to contain four times as many instructions.

2.17.1 [5] <§2.5> How would this affect the size of each of the bit fields in the R-type instructions?

2.17.2 [5] <§2.5> How would this affect the size of each of the bit fields in the I-type instructions?

2.17.3 [5] <§§2.5, 2.8, 2.10> How could each of the two proposed changes decrease the size of a LEGv8 assembly program? On the other hand, how could the proposed change increase the size of an LEGv8 assembly program?

2.18 Assume the following register contents:

```
X10 = 0x00000000AAAAAAAA, X11 = 0x1234567812345678
```

2.18.1 [5] <§2.6> For the register values shown above, what is the value of `X12` for the following sequence of instructions?

```
LSL X12, X10, #4
ORR X12, X12, X11
```

2.18.2 [5] <§2.6> For the register values shown above, what is the value of `X12` for the following sequence of instructions?

```
LSL X12, X11, #4
```

2.18.3 [5] <§2.6> For the register values shown above, what is the value of `X12` for the following sequence of instructions?

```
LSR  X12, X10, #3
ANDI X12, X12, 0xFEF
```

2.19 [10] <§2.6> Find the shortest sequence of LEGv8 instructions that extracts bits 16 down to 11 from register `X10` and uses the value of this field to replace bits 31 down to 26 in register `X11` without changing the other bits of registers `X10` or `X11`. (Be sure to test your code using `X10 = 0` and `X11 = 0xffffffffffffffff`. Doing so may reveal a common oversight.)

2.20 [5] <§2.6> Provide a minimal set of LEGv8 instructions that may be used to implement the following pseudoinstruction:

```
NOT X10, X11        // bit-wise invert
```

2.21 [5] <§2.6> For the following C statement, write a minimal sequence of LEGv8 assembly instructions that performs the identical operation. Assume `X11 = A`, and `X13` is the base address of C.

```
A = C[0] << 4;
```

2.22 [5] <§2.7> Assume `X0` holds the value `0×0000000000101000`. What is the value of `X1` after the following instructions?

```
        CMP X0, #0
        B.GE ELSE
        B DONE
ELSE:   ORRI X1, XZR, #2
DONE:
```

2.23 Suppose the *program counter* (PC) is set to `0×2000 0000`.

2.23.1 [5] <§2.10> What range of addresses can be reached using the LEGv8 *branch* (`B`) instruction? (In other words, what is the set of possible values for the PC after the branch instruction executes?).

2.23.2 [5] <§2.10> What range of addresses can be reached using the LEGv8 *conditional branch-on-equal* (`CBZ`) instruction? (In other words, what is the set of possible values for the PC after the branch instruction executes?)

2.24 Consider a proposed new instruction named `RPT`. This instruction combines a loop's condition check and counter decrement into a single instruction. For example `RPT X12, loop` would do the following:

```
if (X12 >0) {
        X12 = X12 -1;
        goto loop
    }
```

2.24.1 [5] <§2.7, 2.10> If this instruction were to be added to the ARMv8 instruction set, what is the most appropriate instruction format?

2.24.2 [5] <§2.7> What is the shortest sequence of LEGv8 instructions that performs the same operation?

2.25 Consider the following LEGv8 loop:

```
LOOP:  SUBIS X1, X1, #0
       B.LE DONE
       SUBI X1, X1, #1
       ADDI X0, X0, #2
       B LOOP
DONE:
```

2.25.1 [5] <§2.7> Assume that the register `X1` is initialized to the value 10. What is the final value in register `X0` assuming the `X0` is initially zero?

2.25.2 [5] <§2.7> For the loop above, write the equivalent C code. Assume that the registers `X0`, and `X1` are integers `acc` and `i` respectively.

2.25.3 [5] <§2.7> For the loop written in LEGv8 assembly above, assume that the register `X1` is initialized to the value N. How many LEGv8 instructions are executed?

2.25.4 [5] <§2.7> For the loop written in LEGv8 assembly above, replace the instruction "`B.LE DONE`" instruction with "`B.MI DONE`". What is the final value in register `X0` assuming the `X0` is initially zero?

2.25.5 [5] <§2.7> For the loop written in LEGv8 assembly above, replace the instruction "`B.LE DONE`" instruction with "`B.MI DONE`" and write equivalent C code.

2.25.6 [5] <§2.7> What is the purpose of the `SUBIS` instruction in the assembly code above?

2.25.7 [5] <§2.7> Show how you can reduce the number of instructions by combining the `SUBIS` and `SUBI` instructions. (Hint: Add one instruction outside the loop.)

2.26 [10] <§2.7> Translate the following C code to LEGv8 assembly code. Use a minimum number of instructions. Assume that the values of `a`, `b`, `i`, and `j` are in registers `X0`, `X1`, `X10`, and `X11`, respectively. Also, assume that register `X2` holds the base address of the array D.

```
for(i=0; i<a; i++)
   for(j=0; j<b; j++)
      D[4*j] = i + j;
```

2.27 [5] <§2.7> How many LEGv8 instructions does it take to implement the C code from Exercise 2.26? If the variables `a` and `b` are initialized to 10 and 1 and all elements of D are initially 0, what is the total number of LEGv8 instructions executed to complete the loop?

2.28 [5] <§2.7> Translate the following loop into C. Assume that the C-level integer `i` is held in register `X10`, `X0` holds the C-level integer called `result`, and `X1` holds the base address of the integer `MemArray`.

```
      ORR X10, XZR, XZR
LOOP: LDUR X11, [X1, #0]
      ADD X0, X0, X11
      ADDI X1, X1, #8
      ADDI X10, X10, #1
      CMPI X10, 100
      B.LT LOOP
```

2.29 [10] <§2.7> Rewrite the loop from Exercise 2.28 to reduce the number of LEGv8 instructions executed. Hint: Notice that variable `i` is used only for loop control.

2.30 [30] <§2.8> Implement the following C code in LEGv8 assembly. Hint: Remember that the stack pointer must remain aligned on a multiple of 16.

```
int fib(int n){
    if (n==0)
       return 0;
    else if (n == 1)
       return 1;
    else
       return fib(n-1) + fib(n-2);
}
```

2.31 [20] <§2.8> For each function call, show the contents of the stack after the function call is made. Assume the stack pointer is originally at address `0x7ffffffc`, and follow the register conventions as specified in Figure 2.12.

2.32 [20] <§2.8> Translate function `f` into LEGv8 assembly language. If you need to use registers `X10` through `X27`, use the lower-numbered registers first. Assume the function declaration for `g` is "`int g(int a, int b)`". The code for function `f` is as follows:

```
int f(int a, int b, int c, int d){
  return g(g(a,b),c+d);
}
```

2.33 [5] <§2.8> Can we use the tail-call optimization in this function? If no, explain why not. If yes, what is the difference in the number of executed instructions in `f` with and without the optimization?

2.34 [5] <§2.8> Right before your function `f` from Exercise 2.32 returns, what do we know about contents of registers `X5`, `X29`, `X30`, and `SP`? Keep in mind that we know what the entire function `f` looks like, but for function `g` we only know its declaration.

2.35 [30] <§2.9> Write a program in LEGv8 assembly to convert an ASCII string containing a positive or negative integer decimal string to an integer. Your program should expect register `X0` to hold the address of a null-terminated string containing an optional '+' or '−' followed by some combination of the digits 0 through 9. Your program should compute the integer value equivalent to this string of digits, then place the number in register `X0`. If a non-digit character appears anywhere in the string, your program should stop with the value −1 in register `X0`. For example, if register `X0` points to a sequence of three bytes 50_{ten}, 52_{ten}, 0_{ten} (the null-terminated string "24"), then when the program stops, register `X0` should contain the value 24_{ten}. The ARMv8 `MUL` instruction takes two registers as input. There is no "MULI" instruction. Thus, just store the constant 10 in a register.

2.36 Consider the following code:

```
LDURB X10, [X11, #0]
STUR  X10, [X11, #8]
```

Assume that the register `X11` contains the address `0×10000000` and the data at address is `0×1122334455667788`.

2.36.1 [5] <§2.3, 2.9> What value is stored in `0×10000008` on a big-endian machine?

2.36.2 [5] <§2.3, 2.9> What value is stored in `0×10000008` on a little-endian machine?

2.37 [5] <§2.10> Write the LEGv8 assembly code that creates the 64-bit constant $0x1122334455667788_{two}$ and stores that value to register `X0`.

2.38 [10] <§2.11> Write the LEGv8 assembly code to implement the following C code:

```
lock(lk);
shvar=max(shvar,x);
unlock(lk);
```

Assume that the address of the `lk` variable is in `X0`, the address of the `shvar` variable is in `X1`, and the value of variable `x` is in `X2`. Your critical section should not contain any function calls. Use `LDXR/STXR` instructions to implement the `lock()` operation, the `unlock()` operation is simply an ordinary store instruction.

2.39 [10] <§2.11> Repeat Exercise 2.38, but this time use `LDXR/STXR` to perform an atomic update of the `shvar` variable directly, without using `lock()` and `unlock()`. Note that in this exercise there is no variable `lk`.

2.40 [5] <§2.11> Using your code from Exercise 2.38 as an example, explain what happens when two processors begin to execute this critical section at the same time, assuming that each processor executes exactly one instruction per cycle.

2.41 Assume for a given processor the CPI of arithmetic instructions is 1, the CPI of load/store instructions is 10, and the CPI of branch instructions is 3. Assume a program has the following instruction breakdowns: 500 million arithmetic instructions, 300 million load/store instructions, 100 million branch instructions.

2.41.1 [5] <§§1.6, 2.13> Suppose that new, more powerful arithmetic instructions are added to the instruction set. On average, through the use of these more powerful arithmetic instructions, we can reduce the number of arithmetic instructions needed to execute a program by 25%, while increasing the clock cycle time by only 10%. Is this a good design choice? Why?

2.41.2 [5] <§§1.6, 2.13> Suppose that we find a way to double the performance of arithmetic instructions. What is the overall speedup of our machine? What if we find a way to improve the performance of arithmetic instructions by 10 times?

2.42 Assume that for a given program 70% of the executed instructions are arithmetic, 10% are load/store, and 20% are branch.

2.42.1 [5] <§§1.6, 2.13> Given this instruction mix and the assumption that an arithmetic instruction requires two cycles, a load/store instruction takes six cycles, and a branch instruction takes three cycles, find the average CPI.

2.42.2 [5] <§§1.6, 2.13> For a 25% improvement in performance, how many cycles, on average, may an arithmetic instruction take if load/store and branch instructions are not improved at all?

2.42.3 [5] <§§1.6, 2.13> For a 50% improvement in performance, how many cycles, on average, may an arithmetic instruction take if load/store and branch instructions are not improved at all?

2.43 [10] <§2.19> Use the full ARMv8 instruction set (in particular the ability to scale a register offset when loading data) to further reduce the number of assembly instructions needed to carry out the function given in Exercise 2.4.

2.44 [10] <§2.19> Use the full ARMv8 instruction set (in particular the ability to scale a register offset when loading data) to further reduce the number of assembly instructions needed to implement the C code given in Exercise 2.8.

2.45 [10] <§2.19> Use the full ARMv8 instruction set to further reduce the number of instructions needed to complete Exercise 2.19.

2.46 [10] <§2.19> Provide a minimal set of ARMv8 instructions that may be used to implement the following pseudoinstruction:

```
NOT X10, X11        // bit-wise invert
```

(Note that some LEGv8 answers to Exercise 2.20 may not be valid ARMv8.)

2.47 [10] <§2.19> Use the pre-indexed addressing mode to minimize the number of ARMv8 assembly instructions needed to implement the code given in Exercise 2.28.

2.48 [5] <§2.19> Use the post-indexed addressing mode to minimize the number of ARMv8 assembly instructions needed to implement the code given in Exercise 2.28.

2.49 [5] <§2.19> Implement an ARMv8 assembly function to find the maximum value in an integer array. Optimize your function to require as few instructions as possible. Hint: using the `CSEL` instruction, it is possible to reduce the body of the loop to five instructions.

Answers to Check Yourself

§2.2, page 67: ARMv8, C, Java.
§2.3, page 73: 2) Very slow.
§2.4, page 81: 2) -8_{ten}
§2.5, page 89: 3) `SUB X11, X10, X9`
§2.6, page 92: Both. AND with a mask pattern of 1s will leaves 0s everywhere but the desired field. Shifting left by the correct amount removes the bits from the left of the field. Shifting right by the appropriate amount puts the field into the rightmost bits of the doubleword, with 0s in the rest of the doubleword. Note that AND leaves the field where it was originally, and the shift pair moves the field into the rightmost part of the doubleword.
§2.7, page 99: I. All are true. II. 1).
§2.8, page 110: Both are true.
§2.9, page 115: I. 1) and 2) II. 3).
§2.10, page 124: I. 4) ±1024 K. II. 4) ± 128 M.
§2.11, page 128: Both are true.
§2.12, page 137: 4) Machine independence.

This page intentionally left blank